GQA Implementation Project — Detailed Step-by-Step Plan

PHASE 1: Naive GQA Baseline (Student A)

Overview

Build correctness-first. Start in pure PyTorch to nail down the math, then port to a basic CUDA kernel with minimal but meaningful optimizations.

Step 1 — Environment Setup & Background Reading (Day 1)

Goal: Get your dev environment ready and understand GQA at the math level before writing a line of code.

- [ ] Install PyTorch with CUDA support (`torch`, `triton`, `ninja` for custom extensions)
- [ ] Set up a CUDA-capable environment (local GPU or Colab A100/T4)
- [ ] Install `torch.utils.cpp_extension` for building custom CUDA kernels inline
- [ ] Read the original MQA/GQA paper: "*GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*" (Ainslie et al., 2023)
- [ ] Write down in your own words: how GQA differs from MHA and MQA, what "grouping" means, and how KV heads are shared across Q heads

Checkpoint: You should be able to answer — if I have 8 Q heads and 2 KV heads, what is the group size, and which K/V head does Q head 5 attend to?

Step 2 — Pure PyTorch Reference Implementation (Days 2–3)

Goal: A clean, readable, mathematically correct GQA forward pass in PyTorch. This is your ground truth for all future correctness checks.

File: `gqa_reference.py`

- [ ] Define input shapes: `Q [B, H_q, S, D]`, `K [B, H_kv, S, D]`, `V [B, H_kv, S, D]`
  - `B` = batch size, `H_q` = query heads, `H_kv` = KV heads, `S` = sequence length, `D` = head dim
- [ ] Implement the KV head expansion/repeat logic:

```python
```

```
# group_size = H_q // H_kv
K_expanded = K.repeat_interleave(group_size, dim=1)  # [B, H_q, S, D]
V_expanded = V.repeat_interleave(group_size, dim=1)
```

- [ ] Implement scaled dot-product attention:

```python
scores = torch.einsum('bhsd,bhtd->bhst', Q, K_expanded) / math.sqrt(D)
attn = torch.softmax(scores, dim=-1)
out = torch.einsum('bhst,bhtd->bhsd', attn, V_expanded)
```

- [ ] Add an optional causal mask (upper-triangular masking)
- [ ] Write a unit test that compares your output against `F.scaled_dot_product_attention` with repeated KV heads
- [ ] Verify output shapes, dtype, and numerical values match to within 1e-5

Checkpoint: Unit tests pass. You have a reference you trust completely.

Step 3 — Memory Layout & Access Pattern Analysis (Day 4)

Goal: Before writing CUDA, understand exactly how data is laid out in memory and what the ideal access pattern looks like.

- [ ] Print the memory strides for Q, K, V tensors using `.stride()` — understand row-major vs column-major implications
- [ ] Draw (on paper or a diagram tool) the tiling strategy:
  - Tile Q along the sequence dimension by block size `Br`
  - Tile K/V along the sequence dimension by block size `Bc`
  - Each thread block computes one output tile
- [ ] Identify the memory bottleneck: for a naive implementation, K and V are read once per Q tile — calculate the total HBM reads in GB for a typical config (e.g., S=2048, D=64, H_kv=8, B=4)
- [ ] Document which dimensions are "hot" (accessed repeatedly) and which are "cold" — this will drive your shared memory strategy
- [ ] Sketch the thread block → warp → thread assignment before writing any CUDA

Step 4 — Basic CUDA Kernel: QK^T Matmul (Days 5–7)

Goal: Write a correct CUDA kernel for the Q×K^T computation with basic shared memory tiling.

File: `gqa_cuda_naive.cu`

- [ ] Set up the CUDA extension build system using `torch.utils.cpp_extension.load()`
- [ ] Write the kernel signature:

```c
__global__ void gqa_forward_kernel(
    float* Q, float* K, float* V, float* O,
    int B, int H_q, int H_kv, int S, int D,
    int group_size
)
```

- [ ] Implement shared memory tiling for Q and K tiles:

```c
__shared__ float Q_shared[Br][D];
__shared__ float K_shared[Bc][D];
```

- [ ] Load Q tile into shared memory (coalesced — threads load consecutive memory addresses)
- [ ] Implement the KV head index mapping inside the kernel: `int kv_head = q_head / group_size;`
- [ ] Compute `QK^T` for the tile, accumulate into a register-level accumulator
- [ ] Write a correctness test comparing CUDA kernel output to your PyTorch reference (tolerance 1e-4 for float32)

Common pitfalls to watch for:

- Stride mismatches when indexing multi-dimensional tensors in flat CUDA memory
- Missing `__syncthreads()` after shared memory loads
- Off-by-one errors in the KV head mapping

Step 5 — Add Softmax and V Accumulation (Days 8–9)

Goal: Complete the attention computation — softmax over the score row, then weighted sum over V.

- [ ] Implement a row-wise softmax in the kernel (each thread handles one row of the attention score matrix):
  - Find row max (for numerical stability)
  - Subtract max, exponentiate
  - Sum, then divide
- [ ] Load V tile into shared memory: `__shared__ float V_shared[Bc][D];`

- [ ] Accumulate `attn_weights × V` into output buffer
- [ ] Fuse the V accumulation into the same kernel pass (don't write intermediate attention weights to HBM)
- [ ] Add causal masking: set scores to `-inf` where `col > row` before softmax
- [ ] Run correctness tests again — numerical tolerance may need to widen slightly to 1e-3 due to softmax accumulation order differences

---

Step 6 — Thread Block Organization & Coalescing Audit (Day 10)

Goal: Ensure memory accesses are coalesced and your thread/block dimensions are tuned.

- [ ] Profile the kernel with `ncu` (Nsight Compute): check "Memory Throughput" and "L2 Hit Rate"
- [ ] Verify that consecutive threads access consecutive memory addresses for Q, K, V loads (coalesced = full cache line used)
- [ ] Tune `Br` and `Bc` tile sizes (try 32, 64, 128) — measure with `torch.cuda.Event` timing
- [ ] Ensure thread block dimensions are multiples of 32 (warp size)
- [ ] Document your findings: what tile size gave best performance and why

---

Step 7 — Benchmarking & Documentation (Day 11)

Goal: Quantify your implementation's performance and summarize learnings.

- [ ] Benchmark against PyTorch's `F.scaled_dot_product_attention` across sequence lengths: 512, 1024, 2048, 4096
- [ ] Measure peak memory usage using `torch.cuda.max_memory_allocated()`
- [ ] Record latency (ms) and throughput (TFLOPS) for each config
- [ ] Write a 1-page summary covering:
  - GQA mechanics and the group_size concept
  - Why memory coalescing matters and what you did to achieve it
  - What bottleneck remains (hint: softmax prevents full reuse of loaded tiles)

PHASE 2: Flash-Attention Style GQA (Student B)

Overview

Fix the fundamental bottleneck from Phase 1 — the attention matrix. Flash Attention avoids materializing the full `[S, S]` attention matrix by using online softmax and tiled computation entirely in SRAM. Adapt this to GQA.

Step 1 — Environment Setup & Deep Background Reading (Day 1)

Goal: Understand the Flash Attention algorithm at the algorithmic level before touching code.

- [ ] Read Flash Attention v1 paper carefully: "*FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*" (Dao et al., 2022)
- [ ] Read the online softmax algorithm: "*Online normalizer calculation for softmax*" (Milakov & Gimelshein, 2018)
- [ ] Understand the key insight: you can compute softmax incrementally if you track running `(max, sum)` statistics
- [ ] Read about the backward pass recomputation trick: instead of storing the attention matrix for the backward pass, recompute it from Q, K during backprop
- [ ] Install dependencies: same CUDA environment as Phase 1, plus optionally `flash-attn` package as a correctness reference
- [ ] Write pseudocode for Flash Attention's forward pass tiling loop before writing real code

Checkpoint: Be able to explain to someone else why Flash Attention reads HBM O(S) times instead of $O(S^2)$ times.

Step 2 — PyTorch Reference with Online Softmax (Days 2–3)

Goal: Implement the online softmax logic in pure Python/PyTorch to validate your understanding before moving to CUDA.

File: `flash_gqa_reference.py`

- [ ] Implement online softmax accumulation in Python (tile by tile, updating running max and sum):

```python
m_i = -inf  # running max
l_i = 0.0   # running sum of exp(x - max)
O_i = zeros(D)

for each K/V tile j:
    s_ij = Q_i @ K_j.T / sqrt(D)   # [Br, Bc] scores
    m_new = max(m_i, row_max(s_ij))
    l_new = exp(m_i - m_new) * l_i + sum(exp(s_ij - m_new))
    O_i = (exp(m_i - m_new) * l_i * O_i + exp(s_ij - m_new) @ V_j) / l_new
    m_i, l_i = m_new, l_new
```

- [ ] Verify this produces identical output to your Phase 1 PyTorch reference (tolerance 1e-5)
- [ ] Integrate GQA KV head mapping: use `kv_head = q_head // group_size` when selecting K/V tiles
- [ ] Add causal masking inside the tile loop (mask out future positions within each tile)

Checkpoint: Output matches Phase 1 reference exactly. You trust the online softmax logic.

Step 3 — HBM Access Analysis & Tiling Plan (Day 4)

Goal: Quantify exactly how much HBM bandwidth you'll save vs Phase 1, and plan the CUDA tiling.

- [ ] Calculate HBM reads for Phase 1 naive approach:
  - Q: read once = `B × H_q × S × D × 4 bytes`
  - K, V: read once per Q tile = `B × H_kv × S × D × 4 bytes × (S / Br)` times
  - Attention matrix: written and read = `B × H_q × S × S × 4 bytes`
- [ ] Calculate HBM reads for Flash Attention approach:
  - Q: read `S/Br` times (one tile at a time)
  - K, V: read `S/Bc` times per Q tile = `O(S²/SRAM_size)` total — but crucially **no attention matrix written to HBM**
- [ ] For a typical config (S=2048, D=64, H_q=32, H_kv=4), calculate the savings in GB
- [ ] Determine SRAM budget: A100 has 192KB shared memory per SM — plan your `Br` and `Bc` to fit `Q_tile + K_tile + V_tile + O_tile + stats` within this budget
- [ ] Draw the full tiling diagram: outer loop over Q tiles, inner loop over KV tiles

---

Step 4 — Fused Forward Kernel: QK^T + Online Softmax + V Accumulation (Days 5–8)

Goal: Write a single CUDA kernel that performs QK^T, online softmax, and weighted V sum in one pass without materializing the attention matrix.

File: `flash_gqa_cuda.cu`

- [ ] Write kernel with the following register/shared memory layout per thread block:

```c
__shared__ float Q_tile[Br][D];   // one Q tile
__shared__ float K_tile[Bc][D];   // one KV tile
__shared__ float V_tile[Bc][D];   // one KV tile
float O_tile[D] = {0};          // per-thread output accumulator (registers)
float m_i = -INFINITY;          // running max (register)
float l_i = 0.0f;               // running sum (register)
```

- [ ] Outer loop: iterate over Q tiles (each thread block handles one Q tile)
- [ ] Inner loop: iterate over KV tiles, loading K and V from HBM to shared memory
  - Map KV head: `int kv_head = q_head / group_size;`
  - Load `K[b, kv_head, kv_start:kv_start+Bc, :]` and `V[...]` into shared memory
- [ ] Compute `S_ij = Q_tile @ K_tile.T / sqrt(D)` using shared memory
- [ ] Apply causal masking: set `S_ij[r, c] = -inf` where global `c > r`
- [ ] Update running statistics and output:

```
c

float m_new = fmaxf(m_i, row_max(S_ij));
float l_new = expf(m_i - m_new) * l_i + row_sum_exp(S_ij - m_new);
O_tile = (expf(m_i - m_new) * l_i * O_tile + exp(S_ij - m_new) @ V_tile) / l_new;
m_i = m_new; l_i = l_new;
```

- [ ] Write the final `O_tile` to HBM after all KV tiles are processed
- [ ] Run correctness tests against Phase 1 output and PyTorch reference

Common pitfalls:

- The `l_new` division for `O_tile` must be done after the final tile, not inside the loop (or you must undo it each iteration — the "rescaling" pattern)
- Warp-level reduction needed for `row_max` and `row_sum` — use `__reduce_max_sync` and `__reduce_add_sync`
- Ensure `__syncthreads()` after loading each K and V tile

Step 5 — Block-Sparse Computation for Causal Masking (Day 9)

Goal: Skip fully-masked KV tiles entirely (when all positions are masked in a causal setting) to achieve block-sparsity.

- [ ] Identify when an entire KV tile can be skipped: if `kv_tile_start > q_tile_end` (all future), skip the tile
- [ ] Add the tile-skip condition at the top of the inner KV loop:

```
c

if (kv_start > q_end) continue;  // entire tile is masked
```

- [ ] For partially-masked tiles (diagonal tiles), continue applying element-wise masking as before

- [ ] Measure the speedup this provides for causal attention — for sequence length S, this roughly halves computation
- [ ] Verify correctness is maintained (output should still match reference exactly)

Step 6 — Backward Pass with Recomputation (Days 10–12)

Goal: Implement the backward pass without storing the attention matrix, using recomputation instead.

- [ ] Understand what gradients need to be computed: `dQ`, `dK`, `dV`
- [ ] In the forward pass, save only: `Q`, `K`, `V`, `O`, and the per-row log-sum-exp `L = m + log(l)` — do NOT save the attention weights
- [ ] In the backward kernel, for each tile:
  - Recompute `S_ij = Q_tile @ K_tile.T / sqrt(D)` from saved Q and K
  - Recompute `P_ij = exp(S_ij - L_i)` (the attention weights for this tile)
  - Compute `dV += P_ij.T @ dO`
  - Compute `dP = dO @ V.T`
  - Compute `dS = P * (dP - delta)` where `delta = rowsum(dO * O)` (the "D" term in Flash Attention paper)
  - Compute `dQ += dS @ K`, `dK += dS.T @ Q`
- [ ] Save `L` (log-sum-exp) and `delta` tensors in the forward pass — these are the only extra memory needed
- [ ] Register the backward kernel using PyTorch's `torch.autograd.Function`
- [ ] Verify gradients match `torch.autograd.gradcheck` with double precision

Step 7 — HBM Access Minimization Audit & Profiling (Day 13)

Goal: Confirm you've actually achieved the IO savings you planned, and find any remaining inefficiencies.

- [ ] Profile with `ncu`: focus on "HBM Read Bandwidth", "L2 Read Hit Rate", and "Shared Memory Efficiency"
- [ ] Confirm the attention matrix is never written to HBM (check memory allocation — $O(S^2)$ allocations should not appear)
- [ ] Measure actual HBM bytes read/written and compare to your theoretical analysis from Step 3
- [ ] Check for shared memory bank conflicts (ncu will report these) — fix by adjusting tile dimensions or padding
- [ ] Profile occupancy: too-large shared memory tiles reduce occupancy; find the sweet spot

---

Step 8 — Benchmarking, Comparison & Documentation (Day 14)

Goal: Quantify your gains and summarize the key algorithmic insights.

- [ ] Benchmark against Phase 1 (naive CUDA), PyTorch SDPA, and if available `flash-attn` package across:
  - Sequence lengths: 512, 1024, 2048, 4096, 8192
  - GQA configs: (H_q=32, H_kv=8), (H_q=32, H_kv=4), (H_q=32, H_kv=1)
- [ ] Measure: latency (ms), peak memory (MB), and throughput (TFLOPS)
- [ ] Create a results table showing Phase 1 vs Phase 2 speedup ratios
- [ ] Write a 2-page technical summary covering:
  - Why standard attention is IO-bound, not compute-bound
  - How online softmax enables tile-by-tile computation
  - Why recomputation is better than storing the attention matrix for long sequences
  - How GQA's KV head grouping interacts with the tiling strategy

---

Cross-Phase Integration Checklist

Once both implementations are complete, run these joint tests:

- [ ] Both implementations produce identical outputs (tolerance 1e-3) for the same inputs
- [ ] Both support the same API: `(Q, K, V, group_size, causal=True/False) → O`
- [ ] Benchmark head-to-head on identical hardware with identical inputs
- [ ] Compare peak memory: Phase 2 should scale as $O(S)$ where Phase 1 scales as $O(S^2)$
- [ ] Write a joint 1-page comparison document summarizing the tradeoffs

---

Key Reference Papers

1. **GQA** — Ainslie et al. 2023: https://arxiv.org/abs/2305.13245
2. **Flash Attention v1** — Dao et al. 2022: https://arxiv.org/abs/2205.14135
3. **Flash Attention v2** — Dao 2023: https://arxiv.org/abs/2307.08691
4. **Online Softmax** — Milakov & Gimelshein 2018: https://arxiv.org/abs/1805.02867
5. **CUDA Programming Guide** — docs.nvidia.com/cuda/cuda-c-programming-guide